



**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering**

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.4

*Dual Actuator Control System —
Binary Relay and Analog PWM*

on Embedded Systems

Checked by:
Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to design and implement a dual-actuator control system on an Arduino Mega 2560 microcontroller. The system combines a binary actuator (relay-driven load) and an analog actuator (PWM-controlled device) under a unified command interface provided by a 4x4 matrix keypad, with real-time status feedback displayed on a 16x2 LCD and structured diagnostic reports on the serial terminal. This corresponds to Varianta C of the lab assignment, which requires simultaneous control of both actuator types with signal conditioning, overload alerting, and FreeRTOS-based multitasking.

The key learning objectives include understanding the principles of binary and analog actuator control, implementing multi-stage signal conditioning pipelines for command signals (saturation, median filtering, EWMA smoothing, and ramping), designing a FreeRTOS-based preemptive multitasking architecture with mutex-protected shared state, and developing modular, reusable software components for actuator drivers, input processing, and display management.

1.2. Problem Definition

The task statement, as given in the lab manual (Section 5.2), is reproduced below in its original Romanian form, followed by the English analysis.

Partea 1 — Control Actuator binar: Pentru un actuator binar bec iluminare prin releu, să se realizeze o aplicație pentru MCU care va interpreta comenzi de control de la utilizator prin STDIO (serial sau keypad), va controla actuatorul afișând starea curentă pe un display LCD.

Partea 2 — Control actuator analogic: Pentru un actuator analogic (ex.: motor DC controlat prin driver L298 sau PWM), să se realizeze o aplicație pentru MCU care va controla periodic actuatorul selectat, va genera alertă de stare și va afișa starea curentă pe un display LCD.

Varianta C (100%): Implementarea sistem de control a doi actuatori, unul binar și unul analogic, cu afișarea valorilor și alertelor de la ambii actuatori pe display LCD.

The combined system must satisfy the following functional requirements:

1. Accept user commands through a 4x4 matrix keypad (not serial terminal) for both actuators.
2. Control a binary actuator via a relay module with software debouncing to prevent false toggling from noisy keypad inputs.
3. Control an analog actuator via PWM with a four-stage signal conditioning pipeline: saturation (clamping to 0–100%), median filtering (impulse noise rejection), EWMA smoothing (fluctuation reduction), and ramping (gradual transitions for actuator protection).

4. Evaluate an overload threshold alert using a hysteresis-based FSM with debounce confirmation.
5. Display both actuator states, conditioned values, and alert status on a 16x2 LCD at 500 ms intervals.
6. Print structured diagnostic reports to the serial terminal every 2 seconds.
7. Implement all functionality using FreeRTOS preemptive tasks with mutex-protected shared state.
8. Respond to user commands with latency under 100 ms.

1.3. Used Technologies

Pulse Width Modulation (PWM)

Pulse Width Modulation is a technique for encoding analog-like behavior using digital signals. A PWM signal alternates between HIGH and LOW at a fixed frequency; the ratio of HIGH time to the total period is called the duty cycle, expressed as a percentage from 0% (always LOW) to 100% (always HIGH). By varying the duty cycle, the average voltage delivered to a load can be smoothly controlled without the need for a digital-to-analog converter.

On the Arduino Mega 2560, PWM is available on pins 2–13 and 44–46, operating at approximately 490 Hz (or 980 Hz on pins 4 and 13). The `analogWrite(pin, value)` function accepts an 8-bit value (0–255) that maps linearly to the duty cycle. PWM is commonly used to control LED brightness, motor speed, and servo position.

Relay Modules

A relay is an electrically operated switch that uses an electromagnetic coil to mechanically close or open a circuit. Relay modules designed for microcontrollers typically include an optocoupler for electrical isolation and a flyback diode for coil protection. The control input requires a simple digital HIGH/LOW signal from a GPIO pin. When activated, the relay closes its Normally Open (NO) contact, connecting the load circuit. When deactivated, the Normally Closed (NC) contact is engaged instead.

Relays enable microcontrollers to switch high-voltage or high-current loads (mains-powered lights, motors, heaters) using only a 5V logic-level signal, providing galvanic isolation between the control and load circuits.

Signal Conditioning for Actuator Commands

Signal conditioning in the context of actuator control refers to the processing of raw user commands before they are applied to the actuator hardware. Unlike sensor signal conditioning (which cleans measurement noise), actuator command conditioning serves to protect the actuator from abrupt changes, reject spurious inputs, and ensure smooth operation.

The four-stage pipeline used in this lab processes each command value as follows. First, saturation clamps the value to the physical limits of the actuator (0–100%). Second, a median filter with a sliding window of 5 samples removes impulsive noise — single erroneous values that could cause unexpected jumps. Third, an Exponentially Weighted Moving Average (EWMA) with $\alpha = 0.4$ smooths the filtered signal, reducing fluctuations while maintaining reasonable responsiveness. Fourth, a ramping stage limits the maximum change per control cycle to 5% per 100 ms, ensuring gradual transitions that protect mechanical actuators from sudden acceleration or deceleration.

FreeRTOS Real-Time Operating System

FreeRTOS is a real-time operating system kernel designed for resource-constrained micro-controllers. It provides preemptive multitasking, where multiple tasks execute concurrently with priority-based scheduling. Higher-priority tasks preempt lower-priority ones, ensuring that time-critical operations (such as input scanning) are serviced promptly.

Inter-task communication in this lab uses a mutex (mutual exclusion semaphore) to protect the shared actuator state structure. Each task acquires the mutex before reading or writing shared data and releases it immediately afterward, preventing data corruption from concurrent access.

PlatformIO Build System

PlatformIO is a professional open-source ecosystem for embedded development. It provides project management, library dependency resolution, multi-board support, and integrated build/upload/monitor tools. In this lab, PlatformIO is used within VS Code to compile, upload, and monitor the application on the Arduino Mega 2560.

Wokwi Simulator

Wokwi is an online and VS Code-integrated simulator for embedded systems. It supports Arduino, ESP32, and other popular platforms with component libraries including relay modules, keypads, LCDs, and LEDs. Wokwi allows testing of the application in a virtual environment before deploying to physical hardware, providing interactive component manipulation and a simulated serial terminal.

1.4. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is a microcontroller development board based on the ATmega2560 AVR chip. It features 54 digital I/O pins (15 of which support PWM output), 16 analog inputs, 4 hardware UART ports, a 16 MHz crystal oscillator, a USB connection, and 8 KB of SRAM. The board provides ample GPIO and PWM resources for controlling multiple actuators simultaneously while running a FreeRTOS scheduler.

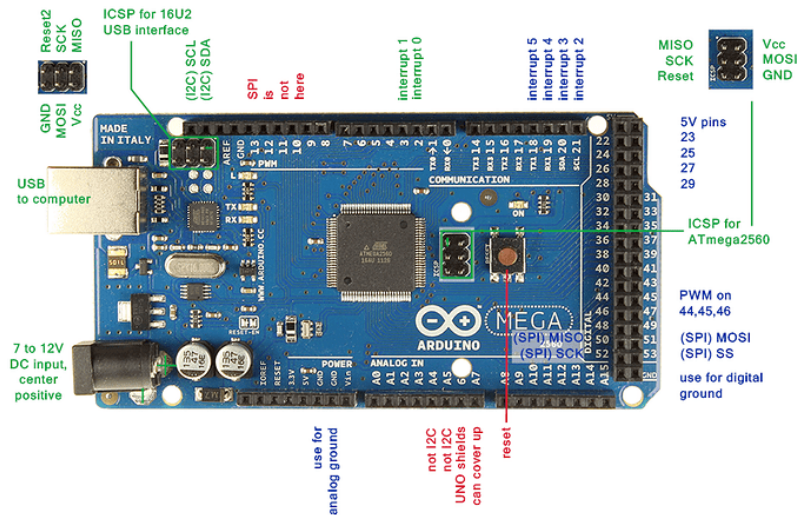


Figure 1.1 – Arduino Mega 2560 development board

Relay Module

The relay module used in this lab is a single-channel 5V relay with an optocoupler for isolation. It provides three output terminals: Normally Open (NO), Normally Closed (NC), and Common (COM). The control input accepts a 5V logic signal from the microcontroller. When the input is driven HIGH, the electromagnetic coil energizes and switches the COM terminal from NC to NO, closing the load circuit.

[Photo pending — save to
resources/DomainAnalysis/HardwareComponents/relay_module.png]

Figure 1.2 – Single-channel 5V relay module with optocoupler

LCD 1602 (I2C)

The LCD 1602 is a 16-column, 2-row character display with an I2C backpack module (PCF8574 I/O expander at address 0x27). The I2C interface reduces the required GPIO connections to just two lines (SDA and SCL), freeing pins for other peripherals. The display is used to show real-time actuator status, conditioned values, and alert indicators.



Figure 1.3 – LCD 1602 display with I2C backpack

4x4 Matrix Keypad

The 4x4 membrane matrix keypad provides 16 keys arranged in a 4-row by 4-column grid. The standard layout includes digits 0–9, symbols * and #, and function keys A–D. The keypad uses 8 GPIO pins (4 rows + 4 columns) and is scanned by setting each row LOW in sequence while reading the column lines to detect pressed keys. Built-in debouncing in the Keypad library ensures reliable detection.

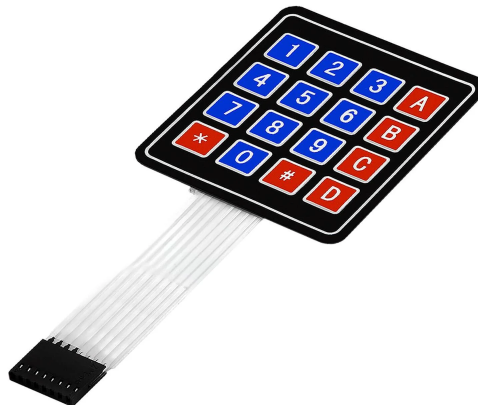


Figure 1.4 – 4x4 membrane matrix keypad

LEDs with Current-Limiting Resistors

Three LEDs are used as visual status indicators: a green LED signals normal system operation, a red LED indicates an overload alert on the analog actuator, and a blue LED (PWM-driven) serves as the simulated analog actuator load, with its brightness reflecting the current duty cycle. Each LED is connected through a 220 Ω current-limiting resistor to prevent excessive current draw from the GPIO pins.

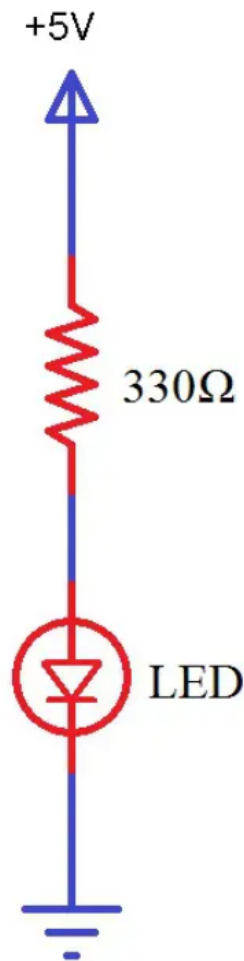


Figure 1.5 – LED indicator with current-limiting resistor

1.5. Software Components

PlatformIO

PlatformIO is used as the primary development environment, integrated within VS Code. It manages the build configuration for multiple lab environments through `platformio.ini`, handles library dependencies (FreeRTOS, LiquidCrystal_I2C, Keypad), and provides serial monitoring for debugging.

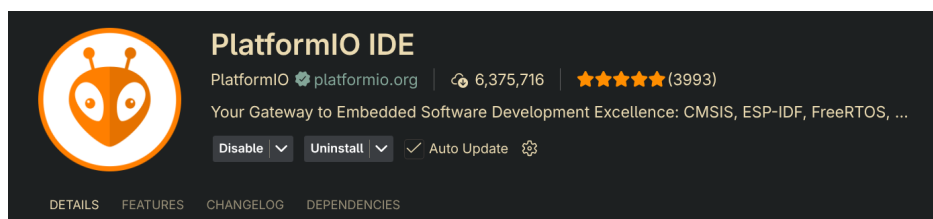


Figure 1.6 – PlatformIO IDE in VS Code

Wokwi

Wokwi is used to simulate the complete circuit including the Arduino Mega 2560, relay module, keypad, LCD, and LEDs. The simulation runs the compiled firmware binary and provides interactive keypad input and visual feedback from the relay, PWM-driven LED, and status LEDs.

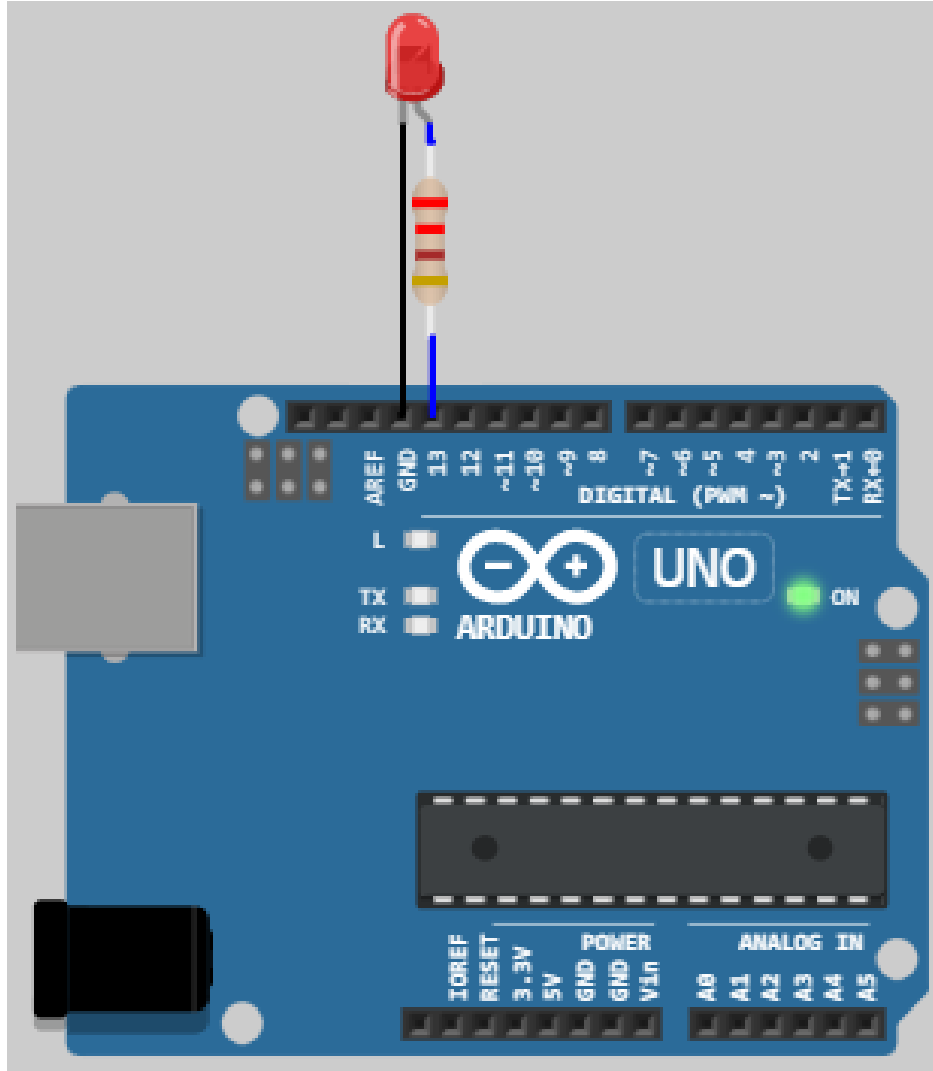


Figure 1.7 – Wokwi simulator running the lab circuit

1.6. System Architecture and Justification

The system follows a layered software architecture that separates concerns across five levels. At the bottom, the Hardware (HW) layer encompasses the physical components: the Arduino Mega 2560 board, relay module, LEDs, keypad, and LCD. Above it, the MCAL (Microcontroller Abstraction Layer) contains the low-level drivers that directly interact with hardware registers — the Relay driver, PwmActuator driver, Led driver, KeypadInput driver, and LcdDisplay driver. The ECAL (ECU Calibration Abstraction Layer) provides higher-level abstractions built on MCAL, including the SignalConditioner pipeline and ThresholdAlert FSM. The SRV (Service Layer) implements the ActuatorConditioner module, which combines signal conditioning with ramping logic. Finally, the APP (Application Layer) contains the three FreeRTOS tasks and the shared state mod-

ule that orchestrate the complete system behavior.

This layered approach was chosen because it enables reuse of driver modules across labs, isolates hardware-specific code from application logic, and allows each layer to be tested independently. The use of FreeRTOS with three dedicated tasks (input at 50 ms, control at 100 ms, display at 500 ms) ensures that time-critical keypad scanning is not blocked by slower LCD updates or serial output.

1.7. Case Study — Relevant Applicability of the Solution

The dual-actuator control pattern implemented in this lab is directly applicable to industrial automation systems. A representative example is a building HVAC (Heating, Ventilation, and Air Conditioning) controller, where binary actuators (relays) control on/off devices like zone dampers and circulation pumps, while analog actuators (PWM or 4–20 mA signals) modulate fan speeds and valve positions. The command conditioning pipeline mirrors real industrial practices: ramping prevents water hammer in valve actuators, median filtering rejects sensor glitches that could cause oscillation, and hysteresis-based alerting avoids alarm flooding from fluctuating measurements near threshold boundaries.

Similar architectures are found in CNC machine controllers (spindle speed via PWM, coolant via relay), agricultural irrigation systems (pump relay + proportional valve), and home automation hubs (light dimming via PWM, appliance switching via relay). The FreeRTOS-based multitasking approach scales naturally from this two-actuator prototype to systems managing dozens of channels, with each actuator’s control loop running as an independent task with defined timing guarantees.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The system integrates six hardware components around the Arduino Mega 2560 microcontroller. The keypad provides user input through a matrix GPIO interface, while the LCD receives status updates via I2C. The relay module is controlled by a single digital GPIO pin, and the PWM actuator (simulated by a blue LED) receives an analog-equivalent signal on a PWM-capable pin. Two status LEDs provide immediate visual feedback: green for normal operation and red for overload alert conditions.

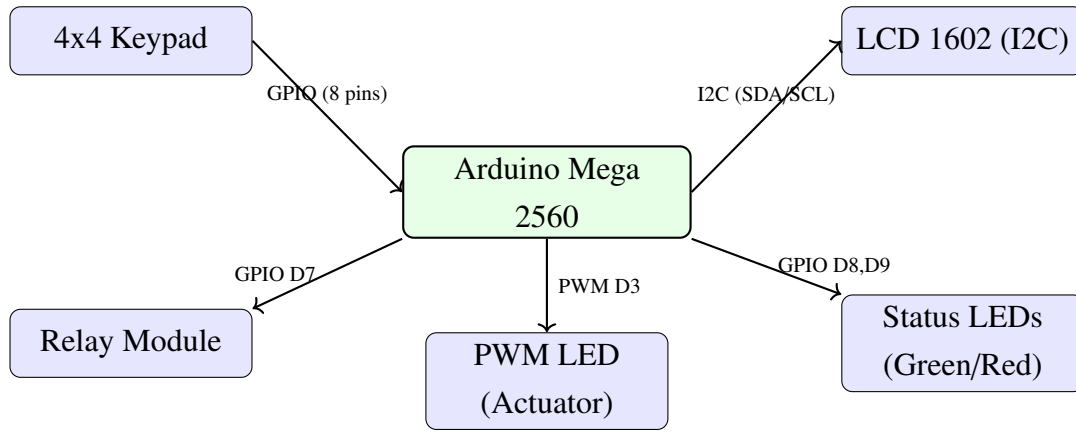


Figure 2.1 – System structural diagram showing all hardware components and interfaces

Layered System Architecture

The software is organized into five layers following the AUTOSAR-inspired embedded systems architecture pattern. Each layer communicates only with its immediate neighbors, enforcing clean separation of concerns and enabling module reuse across labs.

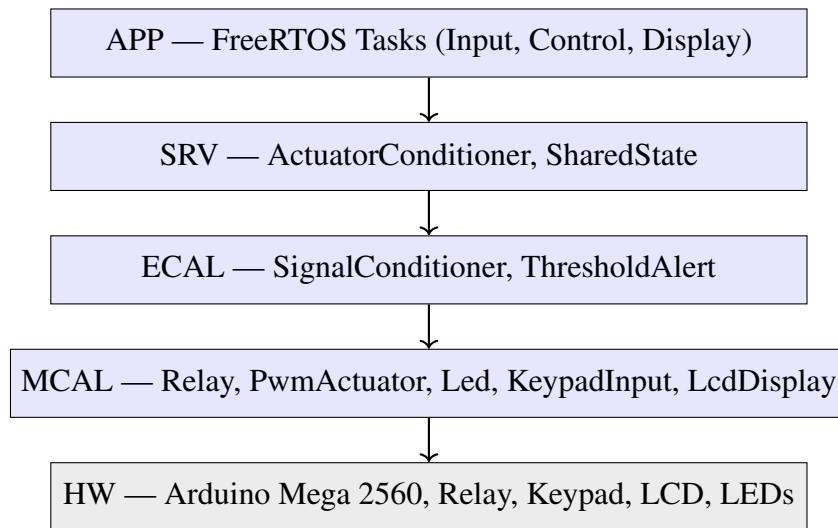


Figure 2.2 – Layered software architecture of the dual-actuator control system

The APP layer contains the three FreeRTOS tasks and the lab entry point. The SRV layer provides the ActuatorConditioner (which wraps the SignalConditioner with ramping logic) and the SharedState module (mutex-protected data exchange). The ECAL layer implements the SignalConditioner pipeline and ThresholdAlert FSM as reusable, hardware-independent modules. The MCAL layer contains all hardware drivers that directly call Arduino GPIO, I2C, and PWM functions. The HW layer represents the physical components.

2.2. Part 1 — Binary Actuator Control (Relay)

This subsection covers the design aspects specific to the binary actuator: the relay-driven load controlled via keypad toggle commands with software debouncing.

Binary Actuator Data Flow

The binary actuator control path is straightforward: the user presses key A on the keypad, which sets a boolean command flag in the shared state. The control task reads this flag, applies software debouncing (3 consecutive confirmations at 100 ms intervals), and drives the relay GPIO pin accordingly. The relay module switches the load circuit through its Normally Open contact.

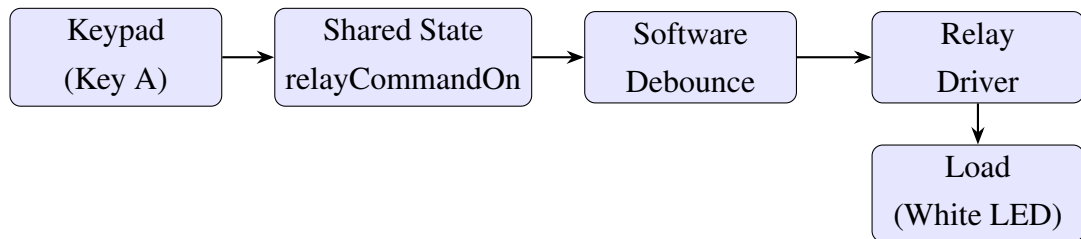


Figure 2.3 – Binary actuator data flow — from keypad to relay load

Relay Debounce Algorithm

The debounce algorithm prevents false relay toggles caused by noisy keypad contacts or accidental double-presses. Rather than debouncing the keypad itself (which the Keypad library already handles), this debounce operates on the *command signal* to ensure that the relay only changes state after the command has been stable for a configurable number of control cycles.

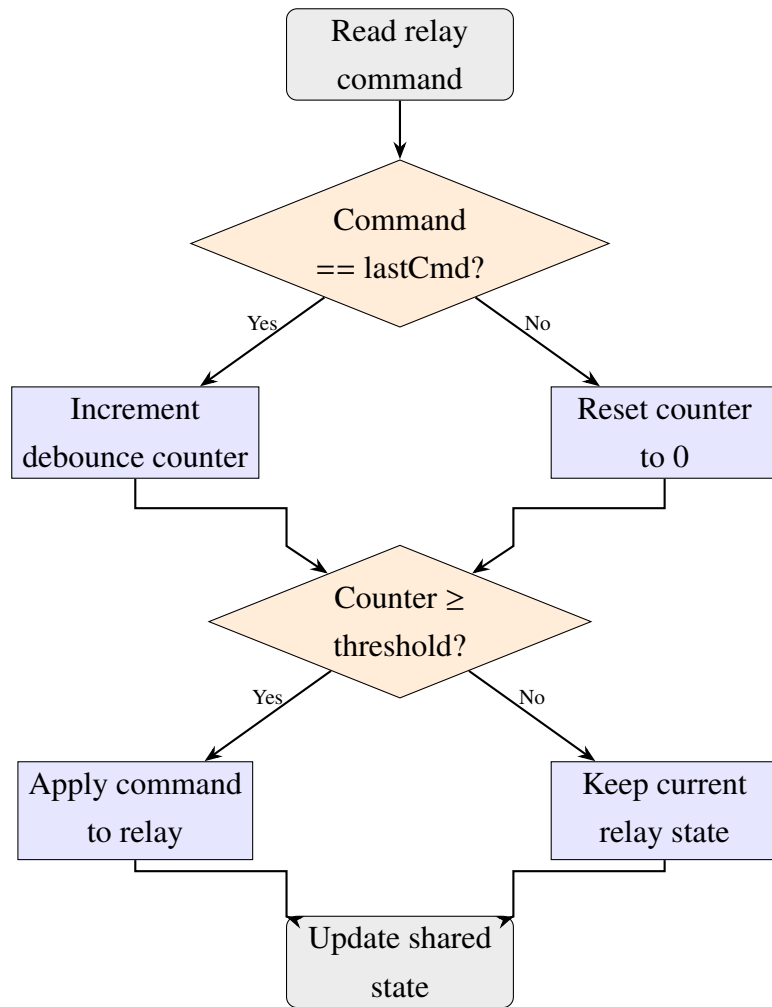


Figure 2.4 – Relay command debounce algorithm flowchart

The debounce threshold is set to 3 confirmations. Since the control task runs every 100 ms, the relay state changes 300 ms after a stable command is received. This delay is imperceptible for human-initiated control while being effective at filtering transient command changes.

Binary Actuator Electrical Schematic

The relay module is connected to pin D7 of the Arduino Mega. When the pin is driven HIGH, the optocoupler inside the module activates the electromagnetic coil, switching the COM terminal from NC to NO. The load LED is connected between COM and ground, with 5V supplied through the NC terminal (which connects to COM when the relay is energized).

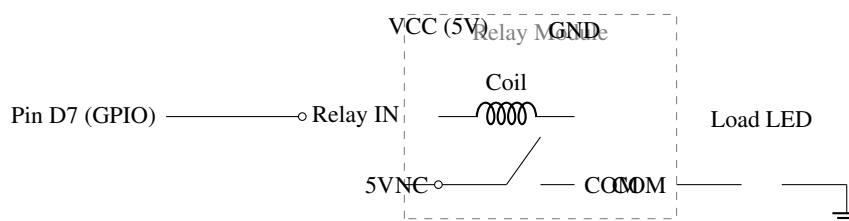


Figure 2.5 – Binary actuator circuit — relay module with load LED

2.3. Part 2 — Analog Actuator Control (PWM)

This subsection covers the design aspects specific to the analog actuator: the PWM-driven load controlled via keypad numeric entry with a four-stage signal conditioning pipeline and overload alerting.

Analog Actuator Data Flow

The analog actuator control path is more complex than the binary path. The user enters a target percentage (0–100) using keys B (enter mode), 0–9 (digits), and # (confirm). The raw command value passes through a four-stage conditioning pipeline before reaching the PWM output. An overload alert FSM monitors the conditioned output and controls the status LEDs.

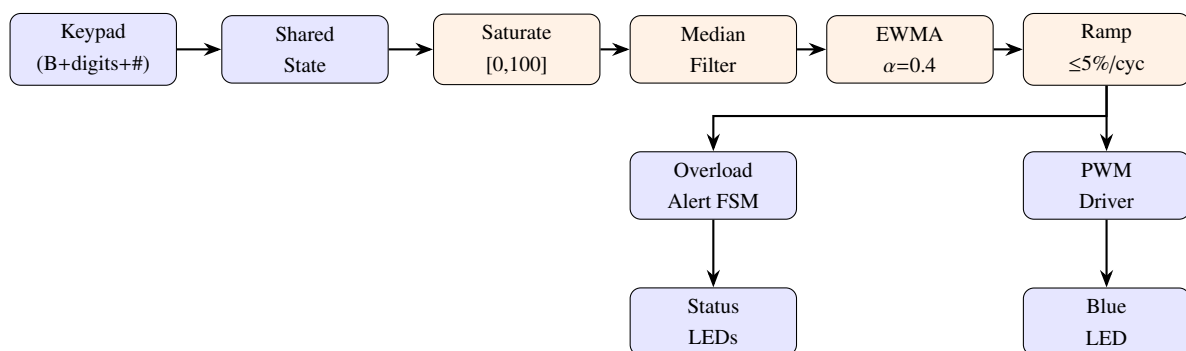


Figure 2.6 – Analog actuator data flow — four-stage conditioning pipeline

Signal Conditioning Pipeline Flowchart

The conditioning pipeline processes each raw command value through four sequential stages. The saturation stage clamps the input to the physical actuator limits (0–100%). The median filter maintains a sliding window of the 5 most recent values and selects the middle one, removing impulsive outliers. The EWMA stage applies exponential smoothing with $\alpha = 0.4$, balancing responsiveness and stability. Finally, the ramping stage limits the output change to 5% per 100 ms cycle, ensuring gradual actuator transitions.

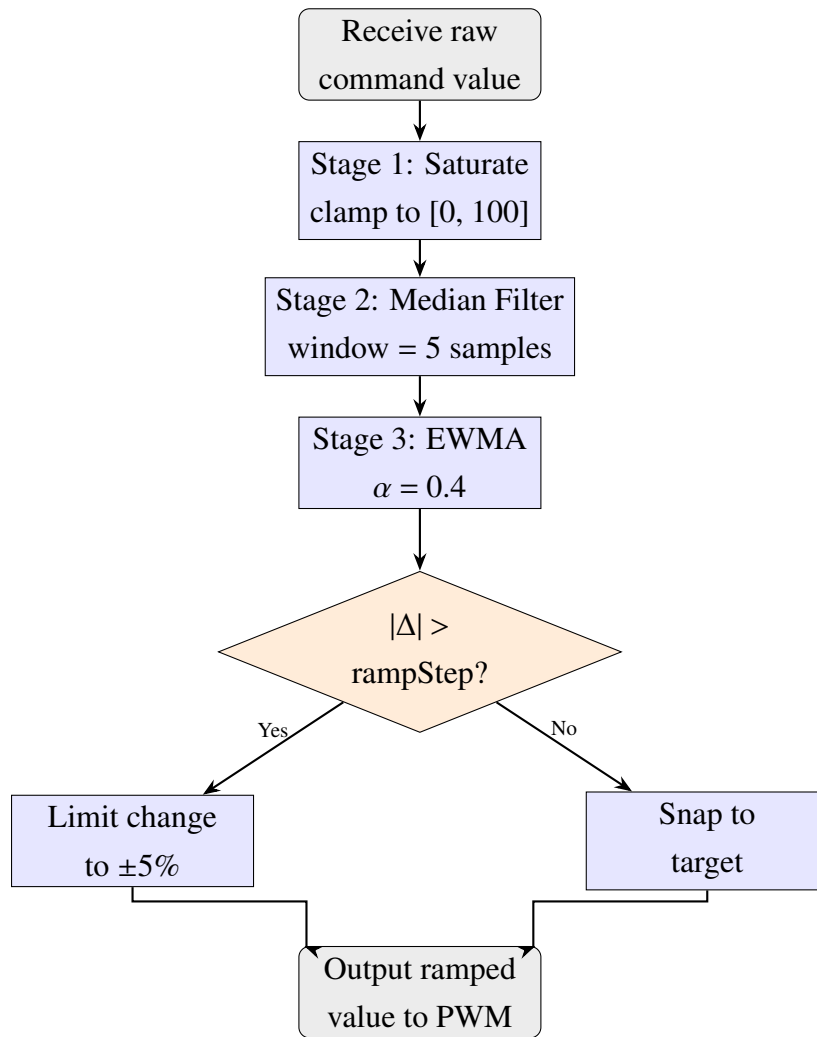


Figure 2.7 – Signal conditioning pipeline flowchart for the analog actuator

Overload Alert FSM

The overload detection uses a 4-state hysteresis FSM with debounce confirmation. An alert triggers when the conditioned PWM output exceeds 90% for 3 consecutive evaluations, and clears when it drops below 80% for 3 consecutive evaluations. The hysteresis band (80%–90%) prevents the alert from oscillating when the actuator operates near the threshold boundary.

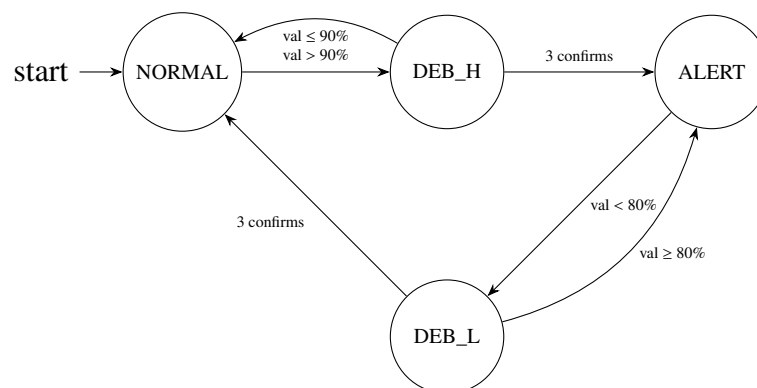


Figure 2.8 – Overload alert FSM state diagram with hysteresis

When the FSM enters the ALERT state, the red LED turns on and the green LED turns off. The LCD display shows !ALERT to indicate the overload condition. Reducing the PWM target below 80% initiates a transition through the DEBOUNCE_LOW state, requiring 3 consecutive sub-threshold readings before returning to NORMAL and restoring the green LED.

Analog Actuator Electrical Schematic

The PWM output on pin D3 drives a blue LED through a $220\ \Omega$ current-limiting resistor. The LED brightness varies proportionally to the duty cycle, visually representing the analog actuator output level. In a real application, the same PWM signal would drive a motor controller (e.g., L298N H-bridge) or a proportional valve driver.

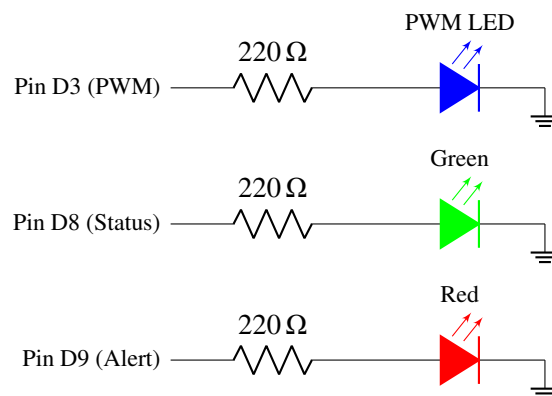


Figure 2.9 – Analog actuator and status LED circuit schematic

2.4. Shared Design Elements

Application Initialization Flowchart

The application starts by initializing the serial interface, shared state, and spawning three FreeRTOS tasks. Once the scheduler starts, each task runs independently at its configured period.

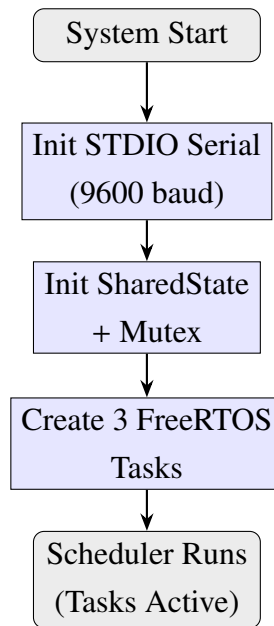


Figure 2.10 – Application initialization flowchart

Input Task Flowchart

The input task scans the keypad every 50 ms and interprets the pressed key. Key A controls the binary actuator, keys B/0–9/# control the analog actuator, C is emergency stop, and D requests an on-demand status report. All key processing is performed inside a mutex-protected critical section to ensure atomicity of shared state updates.

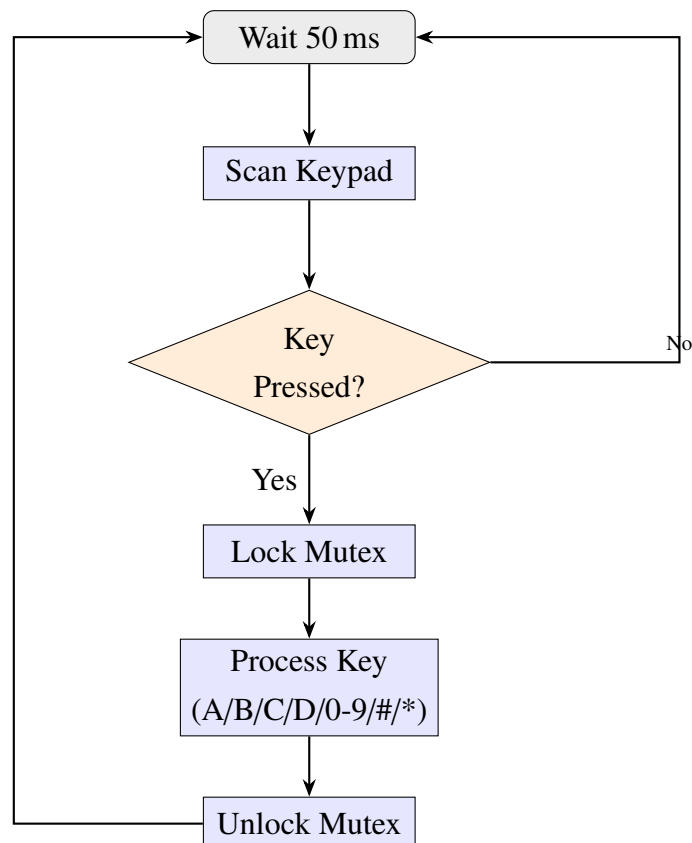


Figure 2.11 – Input task flowchart — keypad scanning and command interpretation

The key mapping is summarized as follows: A toggles the relay ON/OFF (binary actuator), B enters analog input mode, digits 0–9 build the PWM percentage value, # confirms the entered value, * cancels the current input, C triggers an emergency stop that disables both actuators, and D requests an immediate status report on the serial terminal.

Control Task Flowchart

The control task runs every 100 ms and handles both actuator subsystems. It first processes the binary relay with debouncing, then runs the analog conditioning pipeline. Both subsystems share the same mutex-protected critical section to ensure consistent state snapshots.

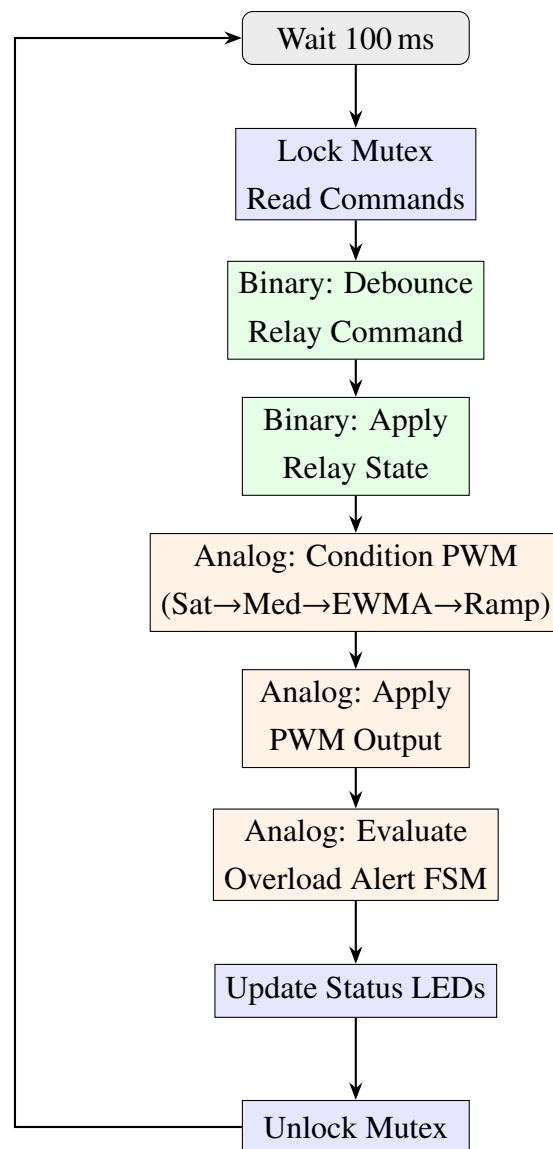


Figure 2.12 – Control task flowchart — binary (green) and analog (orange) processing

2.5. Electrical Schematics

Component Specification

The circuit uses the following components with their electrical specifications:

- **Arduino Mega 2560** — MCU board, 5 V logic, 16 MHz, 54 digital I/O pins
- **Relay module** — 5 V single-channel, optocoupler-isolated, NO/NC/COM terminals
- **LCD 1602 I2C** — 16x2 character display, I2C address 0x27, 5 V supply
- **4x4 Keypad** — membrane matrix, 8 GPIO lines (D22–D29)
- **Blue LED** — PWM actuator indicator, $V_f \approx 3.0$ V, with $220\ \Omega$ resistor
- **Green LED** — system normal indicator, $V_f \approx 2.1$ V, with $220\ \Omega$ resistor
- **Red LED** — overload alert indicator, $V_f \approx 2.0$ V, with $220\ \Omega$ resistor
- **White LED** — relay load indicator (connected through relay NO/COM)

Complete Circuit Connections

The full electrical schematic combines both the binary (relay) and analog (PWM) actuator subsystems. The keypad uses 8 GPIO lines (D22–D29), the LCD connects via I2C (D20/D21), and the relay control signal originates from pin D7. The PWM actuator output uses pin D3, and status LEDs are on pins D8 (green) and D9 (red).

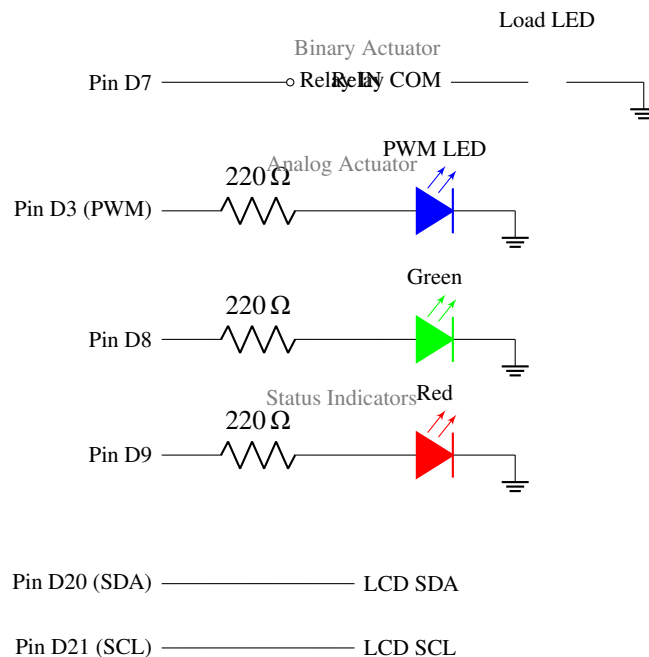


Figure 2.13 – Complete circuit schematic — binary and analog actuator subsystems

Hardware Configuration

The Wokwi simulation configuration uses a breadboard to route connections between the Arduino Mega and peripheral components. The keypad, LEDs with resistors, and power/ground rails all pass through the breadboard, closely resembling a physical prototyping setup.

Listing 2.1 – Wokwi configuration file (wokwi.toml)

```
[wokwi]
version = 1
firmware = "../../.pio/build/lab4/firmware.hex"
elf = "../../.pio/build/lab4/firmware.elf"
```

[Screenshot pending — run the Wokwi simulation and save to
resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.14 – Wokwi simulation circuit with breadboard layout (active run)

2.6. Project Structure

Listing 2.2 – Project directory structure for Lab 4

```
labs/
|-- platformio.ini
|-- src/
|   |-- main.cpp          (lab selector entry point)
|-- lab/
|   |-- lab4/
|       |-- lab4_main.cpp/h  (lab entry point)
|       |-- lab4_config.h    (pin mapping + config)
|       |-- shared_state.cpp/h (mutex-protected shared data)
|       |-- task_input.cpp/h  (keypad scanning task)
|       |-- task_control.cpp/h (actuator control task)
|       |-- task_display.cpp/h (LCD + serial reporting task)
|-- lib/
|   |-- Relay/              (binary actuator driver)
|   |-- PwmActuator/        (analog actuator driver)
|   |-- ActuatorConditioner/ (command conditioning + ramp)
|   |-- SignalConditioner/   (saturate/median/EWMA pipeline)
|   |-- ThresholdAlert/      (hysteresis alert FSM)
|   |-- KeypadInput/         (4x4 keypad driver)
|   |-- LcdDisplay/          (I2C LCD driver)
|   |-- Led/                 (LED driver)
|   |-- StdioSerial/         (STDIO serial redirect)
|-- wokwi/
|   |-- lab4/
|       |-- diagram.json
|       |-- wokwi.toml
```

2.7. Modular Implementation

MCAL Layer: Relay Driver (Binary Actuator)

The Relay module provides a simple interface for controlling a relay connected to a digital GPIO pin. It supports configurable active level (HIGH or LOW) and tracks the current relay state internally.

Listing 2.3 – Relay driver interface (Relay.h)

```
class Relay {
public:
    Relay(uint8_t pin, bool activeHigh = true);
    void init();
    void turnOn();
    void turnOff();
    void toggle();
    void setState(bool on);
    bool isOn() const;
private:
    uint8_t _pin;
    bool _activeHigh;
    bool _state;
};
```

The implementation maps the logical ON/OFF state to the physical pin level based on the activeHigh configuration. When activeHigh is true, turning ON writes HIGH to the pin; when false, it writes LOW. This accommodates both active-HIGH relay modules (common in Arduino ecosystem) and active-LOW modules (common in industrial applications).

MCAL Layer: PWM Actuator Driver (Analog Actuator)

The PwmActuator module abstracts PWM output control, accepting a duty cycle percentage (0–100%) and mapping it to the 8-bit range required by analogWrite().

Listing 2.4 – PwmActuator driver interface (PwmActuator.h)

```
class PwmActuator {
public:
    PwmActuator(uint8_t pin);
    void init();
    void setDuty(float duty);
    float getDuty() const;
    uint8_t getRawPwm() const;
private:
    uint8_t _pin;
    float _duty;
};
```

The `setDuty()` method clamps the input to `[0, 100]` before computing `pwmVal = duty * 255 / 100` and calling `analogWrite()`. This percentage-based interface decouples the application logic from the hardware-specific 8-bit range.

SRV Layer: Actuator Conditioner (Analog Actuator)

The `ActuatorConditioner` module extends the existing `SignalConditioner` with a ramping stage. It wraps the three-stage pipeline (saturation, median filter, EWMA) and adds gradual output transitions to protect mechanical actuators from sudden changes.

Listing 2.5 – ActuatorConditioner interface (ActuatorConditioner.h)

```
class ActuatorConditioner {
public:
    ActuatorConditioner(uint8_t medianWindow, float ewmaAlpha,
                        float minClamp, float maxClamp,
                        float maxRampStep);
    float process(float rawCommand);
    float getRampedOutput() const;
    float getConditionedTarget() const;
    const SignalConditioner& getSignalConditioner() const;
    void reset();
};
```

The ramping logic computes the difference between the conditioned target and the current output, then limits the change to `maxRampStep` per call. This ensures that even if the user commands a jump from 0% to 100%, the actual output ramps up at 5% per 100 ms cycle, reaching full speed in approximately 2 seconds.

ECAL Layer: Threshold Alert FSM (Analog Actuator)

The `ThresholdAlert` module, originally developed in Lab 3 for temperature monitoring, is reused here to detect overload conditions on the analog actuator. The module implements a 4-state hysteresis FSM with configurable high/low thresholds and debounce confirmation count. For this lab, the thresholds are set to 90% (trigger) and 80% (clear) with 3 required confirmations.

APP Layer: Shared State Module

The `SharedState` module defines the `ActuatorState` structure containing all data exchanged between the three tasks. It includes fields for both actuator subsystems: relay command/actual state with debounce counter (binary), and PWM command/conditioned/ramped values with raw register output (analog). A one-shot `reportRequested` flag allows the input task to trigger an immediate serial report via the display task. A FreeRTOS mutex protects concurrent access.

Listing 2.6 – Shared state structure (shared_state.h)

```

struct ActuatorState {
    // Binary actuator
    bool relayCommandOn;
    bool relayActualOn;
    uint8_t relayDebounceCount;
    // Analog actuator
    float pwmCommandPercent;
    float pwmConditioned;
    float pwmRamped;
    uint8_t pwmRawValue;
    // Alert and control
    bool overloadAlert;
    bool inputModeAnalog;
    char inputBuffer[4];
    uint8_t inputBufferLen;
    bool reportRequested;
};

```

APP Layer: FreeRTOS Tasks

Three tasks run concurrently, each with a dedicated period and priority. The Input task (priority 3, 50 ms) has the highest priority to ensure responsive keypad scanning. The Control task (priority 2, 100 ms) processes both binary and analog actuator commands and drives all hardware outputs. The Display task (priority 1, 500 ms) updates the LCD and prints serial reports at a relaxed rate to avoid consuming excessive CPU time with I/O operations.

All tasks follow the same pattern: acquire the mutex, read/write the shared state, release the mutex, then perform hardware I/O outside the critical section where possible. This minimizes mutex hold time and reduces the risk of priority inversion.

3. Obtained Results

3.1. Build Process

The project was compiled using PlatformIO for the lab4 environment targeting the Arduino Mega 2560. The build completed successfully with the following resource utilization: RAM usage of 25.7% (2105 bytes out of 8192 bytes) and Flash usage of 9.8% (24852 bytes out of 253952 bytes). The relatively modest memory footprint leaves ample headroom for future extensions such as additional actuator channels or more complex conditioning algorithms.

[Screenshot pending — save to resources/ObtainedResults/build_results.png]

Figure 3.1 – Successful PlatformIO build output showing RAM and Flash usage

3.2. Simulation Initialization

Upon starting the Wokwi simulation, the system prints a startup banner to the serial terminal listing all configuration parameters: keypad command mapping, hardware pin assignments, conditioning pipeline parameters (median window size, EWMA alpha, ramp step), and overload alert thresholds. The LCD displays the initial state with the relay OFF, PWM at 0%, and no alerts active. The green status LED illuminates immediately, indicating normal system operation.

[Screenshot pending — save to
resources/ObtainedResults/simulation_start.png]

Figure 3.2 – Wokwi simulation startup — LCD showing initial state, serial banner visible

3.3. Part 1 — Binary Actuator (Relay) Results

Test 1.1: Relay Toggle ON (Key A)

Pressing key A on the keypad sends a toggle command to the binary actuator. The serial terminal immediately confirms the command with [INPUT] Relay cmd: ON. After the debounce period (3 consecutive confirmations at 100 ms intervals = 300 ms), the relay module activates: the white load LED illuminates, and the LCD updates its first line to show Relay:ON. The 2-second periodic serial report confirms the relay is in the ON state.

[Screenshot pending — save to resources/ObtainedResults/test_relay_on.png]

Figure 3.3 – Relay ON test — white load LED lit, LCD shows Relay:ON

Test 1.2: Relay Toggle OFF (Key A Again)

Pressing key A a second time sends the OFF command. The serial terminal logs [INPUT] Relay cmd: OFF, and after the 300 ms debounce period, the relay deactivates: the white load LED turns off, and the LCD reverts to Relay:OFF. This confirms the toggle behavior works correctly in both directions with consistent debounce timing.

[Screenshot pending — save to resources/ObtainedResults/test_relay_off.png]

Figure 3.4 – Relay OFF test — load LED off, LCD shows Relay:OFF

Test 1.3: Relay Debounce Validation

To verify that the software debounce prevents false toggles, rapid successive presses of key A are performed. Despite multiple key events arriving within a short window, the relay state changes only once the command has been stable for 3 consecutive 100 ms control cycles. The debounce counter value, visible in the serial report, confirms that transient command changes are filtered out. The relay does not “chatter” (rapidly toggling ON and OFF) during fast keypad interaction.

3.4. Part 2 — Analog Actuator (PWM) Results

Test 2.1: PWM Value Entry (Keys B, 7, 5, #)

Pressing B enters analog input mode. The LCD switches to edit mode, displaying [EDIT] on the first line and the current digit entry on the second. Typing 7 then 5 builds the target percentage “75”, visible both on the LCD and in the serial log ([INPUT] Entering: 7%, [INPUT] Entering: 75%). Pressing # confirms the value, and the serial terminal logs [INPUT] PWM set to 75%. The LCD returns to normal mode showing PWM 75%.

[Screenshot pending — save to
resources/ObtainedResults/test_pwm_entry.png]

Figure 3.5 – PWM value entry — typing 75% via keypad in edit mode

Test 2.2: Signal Conditioning and Ramping

After confirming a target of 75%, the conditioning pipeline processes the command. The saturation stage passes 75 unchanged (within [0, 100]). The median filter requires several cycles to fill its 5-sample window. The EWMA smoothes the transition. Most visibly, the ramping stage limits the output increase to 5% per 100 ms cycle, so the blue PWM LED gradually brightens over approximately 1.5 seconds rather than jumping instantly to 75%.

The serial report shows the pipeline stages: the raw command at 75.0%, the conditioned (post-EWMA) value converging toward 75%, and the ramped output incrementing in 5% steps (0, 5, 10, 15, ... 70, 75). The PWM register value correspondingly increases from 0 to 191 (75% of 255).

[Screenshot pending — save to
resources/ObtainedResults/test_pwm_ramping.png]

Figure 3.6 – PWM ramping — serial report showing gradual output increase to 75%

Test 2.3: Overload Alert (PWM > 90%)

Setting the PWM target to 95% (via B, 9, 5, #) triggers the overload detection sequence. As the ramped output climbs past 90%, the ThresholdAlert FSM transitions from NORMAL to DEBOUNCE_HIGH. After 3 consecutive readings above 90% (300 ms), the FSM enters the ALERT state. The red LED turns on, the green LED turns off, and the LCD displays !ALERT on the second line. The serial report confirms ALERT: OVERLOAD ACTIVE.

[Screenshot pending — save to
resources/ObtainedResults/test_overload_alert.png]

Figure 3.7 – Overload alert active — red LED on, LCD showing !ALERT, serial confirmation

Test 2.4: Alert Hysteresis and Clearing

To verify hysteresis behavior, the PWM target is reduced to 85% (within the hysteresis band of 80–90%). The alert remains active because the FSM only clears when the value drops *below* the low threshold of 80%. Setting the target to 70% causes the ramped output to decrease past 80%, triggering the DEBOUNCE_LOW state. After 3 consecutive sub-80% readings, the FSM returns to NORMAL: the green LED turns back on, the red LED turns off, and the LCD shows OK again. This confirms that the hysteresis band prevents alert oscillation near the threshold boundary.

[Screenshot pending — save to
resources/ObtainedResults/test_alert_clear.png]

Figure 3.8 – Alert cleared after PWM reduced below 80% — green LED restored

Test 2.5: Input Cancellation (Key *)

While in analog input mode (after pressing B and entering partial digits), pressing * cancels the input and returns to normal display mode without changing the PWM target. The serial terminal logs [INPUT] Input cancelled. This verifies that the cancel mechanism prevents accidental actuator changes from incomplete or erroneous input.

3.5. Combined Operation Results

Test 3.1: Simultaneous Binary and Analog Control

Both actuators can be controlled independently within the same session. After setting the relay to ON (key A) and the PWM to 50% (B, 5, 0, #), the LCD shows both states simultaneously: Relay:ON PWM 50% on the first line and Out : 50% OK on the second. The serial report lists both actuator states in a single structured output. The white relay load LED and the dimmed blue PWM LED are both active simultaneously, demonstrating that the FreeRTOS task architecture supports concurrent dual-actuator control without interference.

[Screenshot pending — save to
resources/ObtainedResults/test_combined.png]

Figure 3.9 – Combined operation — relay ON and PWM at 50% simultaneously

Test 3.2: Emergency Stop (Key C)

Pressing key C immediately sets both actuator commands to their safe states: relay OFF and PWM 0%. The serial terminal prints [INPUT] EMERGENCY STOP. The relay deactivates (white LED off), and the PWM ramps down to 0% (blue LED fading). Any active overload alert clears as the output drops below 80%. The LCD returns to the fully-idle state showing Relay:OFF PWM 0% and Out : 0% OK. This provides a one-button safety mechanism for shutting down all actuators simultaneously.

[Screenshot pending — save to
resources/ObtainedResults/test_emergency_stop.png]

Figure 3.10 – Emergency stop — all actuators off, system returned to idle state

Test 3.3: On-Demand Status Report (Key D)

Pressing key D triggers an immediate serial status report without waiting for the 2-second periodic timer. The report includes the trigger source (Trigger: keypad D (on-demand)), both actuator states, the full PWM conditioning pipeline values, and the alert status. This allows rapid diagnostic checks during testing.

[Screenshot pending — save to resources/ObtainedResults/test_report.png]

Figure 3.11 – On-demand serial status report triggered by key D

3.6. System Observations

The system demonstrates responsive command processing, with keypad input detected within 50 ms of the key press. The relay debounce adds a 300 ms delay before the physical relay state changes, which is appropriate for preventing false toggles from noisy membrane contacts. The PWM ramping at 5% per 100 ms produces visibly smooth LED brightness transitions, which would translate to gentle motor speed changes in a real application.

The structured serial reports, printed every 2 seconds (or on demand via key D), provide comprehensive diagnostic information including the raw command, conditioned value, ramped output, PWM register value, and alert state. This level of observability would be valuable for tuning the conditioning parameters in a production system.

Memory utilization remains well within the Arduino Mega 2560's constraints at 25.7% RAM and 9.8% Flash, suggesting the architecture could support significantly more complex applications on the same hardware platform. The FreeRTOS scheduler manages the three tasks without observable timing jitter, and no deadlock or data corruption was encountered during testing.

3.7. Verification Summary

- **Compilation:** Build succeeds with no errors and minimal warnings.
- **Initialization:** Startup banner prints correctly, LCD shows initial state, green LED active.

Part 1 — Binary Actuator:

- **Relay ON/OFF:** Toggle via key A works correctly with LCD and serial confirmation.
- **Debounce:** Relay state only changes after 3 stable command cycles (300 ms).
- **Load indication:** White LED correctly reflects the relay's physical state.

Part 2 — Analog Actuator:

- **Value entry:** Digits build the target value, # confirms, * cancels.
- **Conditioning:** Saturation, median, EWMA pipeline processes commands correctly.
- **Ramping:** Output changes at $\leq 5\%/cycle$, producing smooth transitions.
- **Overload alert:** Triggers at $>90\%$, clears at $<80\%$ with hysteresis and debounce.
- **PWM output:** Blue LED brightness matches the conditioned duty cycle.

Combined:

- **Dual control:** Both actuators operate simultaneously without interference.
- **Emergency stop:** Key C disables all actuators immediately.
- **On-demand report:** Key D triggers immediate serial diagnostic output.
- **Concurrent tasks:** All three FreeRTOS tasks run without deadlock or data corruption.

4. Conclusion

4.1. Achievement of Objectives

The primary objective of implementing a Varianta C dual-actuator control system was fully achieved. The system successfully controls a binary actuator (relay) and an analog actuator (PWM) simultaneously through a unified keypad interface, with real-time LCD feedback and structured serial reporting. All three FreeRTOS tasks operate concurrently at their designated periods without deadlock or data corruption, demonstrating proper use of mutex-protected shared state for inter-task communication.

The four-stage signal conditioning pipeline (saturation, median filter, EWMA, ramping) processes analog commands as specified, producing smooth, protected actuator transitions. The hysteresis-based overload alert FSM with debounce confirmation provides stable, oscillation-free threshold detection.

4.2. Performance Analysis and System Limitations

The system meets the 100 ms latency requirement for command response, with keypad input detected within 50 ms and relay debounce completing in 300 ms. Memory utilization is efficient at 25.7% RAM and 9.8% Flash, leaving substantial headroom for extensions.

The main limitations are: the relay debounce introduces a perceptible 300 ms delay, which is acceptable for safety but could feel sluggish for rapid toggle operations; the PWM ramping at 5% per 100 ms requires 2 seconds for a full 0–100% transition, which may be too slow for some applications (the ramp step is configurable); and the keypad's single-key detection means the system cannot process simultaneous relay and PWM commands.

4.3. Proposed Improvements

Several enhancements could extend the system's capabilities. Adding a PID controller for the analog actuator would enable closed-loop speed or position control with sensor feedback. Implementing a configuration menu accessible through the keypad (using keys to navigate a multi-level LCD menu) would allow runtime adjustment of conditioning parameters without recompilation. Supporting multiple relay channels and PWM outputs would demonstrate scalability. Finally, adding persistent storage (EEPROM) for the last-used actuator settings would improve the user experience across power cycles.

4.4. Reflections on the Learning Experience

This laboratory work provided practical experience with several essential embedded systems concepts. Designing the signal conditioning pipeline for actuator commands, rather than sensor signals, highlighted that the same filtering techniques (median, EWMA) serve dual purposes in embedded control systems. The ramping stage, while simple in implementation, demonstrated the importance of protecting mechanical actuators from abrupt state changes — a principle critical in industrial motor and valve control.

Working with FreeRTOS mutex synchronization reinforced the challenge of concurrent data access in real-time systems. The shared state design pattern, where all inter-task data passes through a single mutex-protected structure, provided a clear and maintainable approach that avoids the complexity of multiple semaphores or message queues for this scale of application.

4.5. Impact of Technology in Real-World Applications

The dual-actuator control pattern is a fundamental building block in industrial automation, building management, and consumer electronics. HVAC systems combine relay-controlled compressors with PWM-modulated fan speeds. Automotive systems use relay-switched fuel pumps alongside PWM-controlled throttle bodies. Manufacturing lines employ relay-actuated pneumatic valves together with PWM-driven servo motors.

The signal conditioning and hysteresis alerting techniques implemented here are standard practice in SCADA (Supervisory Control and Data Acquisition) systems, where reliable actuator control and false-alarm prevention are critical for operational safety and efficiency. The modular, FreeRTOS-based architecture demonstrated in this lab scales directly to industrial controllers managing dozens of actuator channels with deterministic timing guarantees.

5. Note Regarding Usage of AI

During the preparation of this laboratory report, the author utilized artificial intelligence tools to enhance the quality and presentation of the documentation. Specifically, the following AI tools were employed:

- **Report Formatting:** Assistance with LaTeX document structure, formatting consistency, and adherence to academic writing standards.
- **Report Template Generation:** Creation of the initial document template with proper sectioning and organizational structure.
- **Content Rephrasing:** Refinement of technical descriptions and explanations to ensure grammatical accuracy and clarity without altering the technical substance.
- **Code Documentation:** Generation of inline comments within the source code to improve code readability and maintainability.
- **Development Environment Setup:** Configuration assistance for PlatformIO integration with the Wokwi simulation platform.

All AI-generated content was thoroughly reviewed, validated, and adjusted by the author to ensure technical accuracy, alignment with project requirements, and adherence to embedded systems best practices. The core technical implementation, circuit design, system architecture decisions, and experimental results represent the author's original work and understanding of the subject matter. The AI tools served as assistive technologies to enhance documentation quality and development workflow efficiency, while the author retained full responsibility for the content, correctness, and conclusions presented in this report.

6. Bibliography

- [1] Bragarenco, A., Astafi, V., *Sisteme Electronice Încorporate: Indicații metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chișinău, 2024.
- [2] Technical University of Moldova, *Embedded Systems: Practical Guide for Hardware-Software Interface Development*, Department of Microelectronics and Biomedical Engineering, Chișinău, 2024.
- [3] Arduino, *Arduino Mega 2560 Rev3 Documentation*, <https://docs.arduino.cc/hardware/mega-2560/>, Accessed: 2026-03-29.
- [4] PlatformIO, *Professional collaborative platform for embedded development*, <https://platformio.org/>, Accessed: 2026-03-29.
- [5] Wokwi, *Online Electronics Simulator — Arduino, ESP32, Raspberry Pi Pico*, <https://wokwi.com/>, Accessed: 2026-03-29.
- [6] FreeRTOS, *FreeRTOS Real-Time Operating System for Microcontrollers*, <https://www.freertos.org/>, Accessed: 2026-03-29.
- [7] Arduino, *analogWrite() Reference — PWM Output*, <https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/>, Accessed: 2026-03-29.

7. Appendix

The complete source code is organized below by architectural layer. All source files are available in the project GitHub repository.

GitHub Repository: <https://github.com/mcittkmims/es-labs>

7.1. Application Layer

Lab Selector Entry Point

Listing 7.1 – src/main.cpp — Lab selector (Lab 4 section)

```
#elif defined(LAB4)
    #include "lab4_main.h"
// In setup():
    lab4Setup();
// In loop():
    lab4Loop();
```

Lab 4 Entry Point

Listing 7.2 – lab4_main.h — Lab 4 entry point interface

```
#ifndef LAB4_MAIN_H
#define LAB4_MAIN_H
void lab4Setup();
void lab4Loop();
#endif
```

Listing 7.3 – lab4_main.cpp — Lab 4 entry point implementation

```
#include "lab4_main.h"
#include "lab4_config.h"
#include "shared_state.h"
#include "task_input.h"
#include "task_control.h"
#include "task_display.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <stdio.h>
#include "StdioSerial.h"

void lab4Setup() {
    stdioSerialInit(9600);
    // Print startup banner (omitted for brevity)
    sharedStateInit();
    xTaskCreate(vTaskInput, "Input",
```

```

    TASK_INPUT_STACK, NULL, TASK_INPUT_PRIORITY, NULL);
xTaskCreate(vTaskControl, "Control",
    TASK_CONTROL_STACK, NULL, TASK_CONTROL_PRIORITY, NULL);
xTaskCreate(vTaskDisplay, "Display",
    TASK_DISPLAY_STACK, NULL, TASK_DISPLAY_PRIORITY, NULL);
}

void lab4Loop() { /* FreeRTOS tasks handle all logic */ }

```

Configuration and Pin Mapping

Listing 7.4 – lab4_config.h — Pin mapping and configuration

```

#ifndef LAB4_CONFIG_H
#define LAB4_CONFIG_H
#include <Arduino.h>
// Pin assignments
static const uint8_t PIN_RELAY      = 7;
static const uint8_t PIN_PWM_ACT    = 3;
static const uint8_t PIN_LED_GREEN  = 8;
static const uint8_t PIN_LED_RED    = 9;
static byte KEYPAD_ROW_PINS[4] = {22, 23, 24, 25};
static byte KEYPAD_COL_PINS[4] = {26, 27, 28, 29};
static const uint8_t LCD_I2C_ADDR = 0x27;
// Timing
static const uint16_t TASK_INPUT_PERIOD_MS    = 50;
static const uint16_t TASK_CONTROL_PERIOD_MS  = 100;
static const uint16_t TASK_DISPLAY_PERIOD_MS  = 500;
// FreeRTOS task config
static const uint16_t TASK_INPUT_STACK    = 256;
static const uint16_t TASK_CONTROL_STACK  = 256;
static const uint16_t TASK_DISPLAY_STACK  = 512;
static const uint8_t TASK_INPUT_PRIORITY   = 3;
static const uint8_t TASK_CONTROL_PRIORITY = 2;
static const uint8_t TASK_DISPLAY_PRIORITY = 1;
// Conditioning parameters
static const uint8_t ACT_MEDIAN_WINDOW = 5;
static const float   ACT_EWMA_ALPHA    = 0.4f;
static const float   ACT_RAMP_STEP      = 5.0f;
static const float   OVERLOAD_THRESHOLD_HIGH = 90.0f;
static const float   OVERLOAD_THRESHOLD_LOW  = 80.0f;
#endif

```

Shared State Module

Listing 7.5 – shared_state.h — Shared state interface

```

struct ActuatorState {

```

```

    bool relayCommandOn, relayActualOn;
    uint8_t relayDebounceCount;
    float pwmCommandPercent, pwmConditioned, pwmRamped;
    uint8_t pwmRawValue;
    bool overloadAlert, inputModeAnalog;
    char inputBuffer[4];
    uint8_t inputBufferLen;
};
void sharedStateInit();
void sharedStateLock();
void sharedStateUnlock();
ActuatorState* sharedStateGet();

```

Input Task

Listing 7.6 – task_input.cpp — Keypad input task

```

#include "task_input.h"
#include "shared_state.h"
#include "lab4_config.h"
#include "KeypadInput.h"
#include <stdio.h>
#include <stdlib.h>

static KeypadInput keypad(KEYPAD_ROW_PINS, KEYPAD_COL_PINS);

void vTaskInput(void *pvParameters) {
    keypad.init();
    TickType_t xLastWake = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(TASK_INPUT_PERIOD_MS);
    for (;;) {
        char key = keypad.getKey();
        if (key != 0) {
            sharedStateLock();
            ActuatorState *s = sharedStateGet();
            switch (key) {
                case 'A': s->relayCommandOn = !s->relayCommandOn; break;
                case 'B': s->inputModeAnalog = true;
                    s->inputBufferLen = 0; break;
                case 'C': s->relayCommandOn = false;
                    s->pwmCommandPercent = 0.0f; break;
                case '#': if (s->inputModeAnalog && s->inputBufferLen > 0) {
                    s->inputBuffer[s->inputBufferLen] = '\0';
                    int val = atoi(s->inputBuffer);
                    if (val > 100) val = 100;
                    s->pwmCommandPercent = (float)val;
                    s->inputModeAnalog = false;
                } break;
            }
        }
        vTaskDelay(xPeriod);
    }
}

```

```

        case '*': s->inputBufferLen = 0;
                s->inputModeAnalog = false; break;
        default:
                if (key >= '0' && key <= '9' && s->inputModeAnalog)
                        if (s->inputBufferLen < 3)
                                s->inputBuffer[s->inputBufferLen++] = key;
                        break;
                }
        sharedStateUnlock();
    }
    vTaskDelayUntil(&xLastWake, xPeriod);
}
}

```

Control Task

Listing 7.7 – task_control.cpp — Actuator control task

```

#include "task_control.h"
#include "shared_state.h"
#include "lab4_config.h"
#include "Relay.h"
#include "PwmActuator.h"
#include "ActuatorConditioner.h"
#include "ThresholdAlert.h"
#include "Led.h"

static Relay relay(PIN_RELAY, true);
static PwmActuator pwmAct(PIN_PWM_ACT);
static Led ledGreen(PIN_LED_GREEN);
static Led ledRed(PIN_LED_RED);
static ActuatorConditioner conditioner(
    ACT_MEDIAN_WINDOW, ACT_EWMA_ALPHA,
    ACT_MIN_CLAMP, ACT_MAX_CLAMP, ACT_RAMP_STEP);
static ThresholdAlert overloadAlert(
    OVERLOAD_THRESHOLD_HIGH, OVERLOAD_THRESHOLD_LOW,
    OVERLOAD_DEBOUNCE_COUNT);

void vTaskControl(void *pvParameters) {
    relay.init(); pwmAct.init();
    ledGreen.init(); ledRed.init();
    overloadAlert.init(); ledGreen.turnOn();
    TickType_t xLastWake = xTaskGetTickCount();
    for (;;) {
        sharedStateLock();
        ActuatorState *s = sharedStateGet();
        // Relay debounce + control
        relay.setState(s->relayCommandOn);
    }
}

```

```

    s->relayActualOn = relay.isOn();
    // PWM conditioning pipeline
    float output = conditioner.process(s->pwmCommandPercent);
    pwmAct.setDuty(output);
    s->pwmRamped = output;
    s->pwmRawValue = pwmAct.getRawPwm();
    // Overload alert
    overloadAlert.update(output);
    s->overloadAlert = overloadAlert.isAlertActive();
    // LED indicators
    if (s->overloadAlert) { ledRed.turnOn(); ledGreen.turnOff(); }
    else { ledRed.turnOff(); ledGreen.turnOn(); }
    sharedStateUnlock();
    vTaskDelayUntil(&xLastWake,
        pdMS_TO_TICKS(TASK_CONTROL_PERIOD_MS));
}
}

```

Display Task

Listing 7.8 – task_display.cpp — LCD and serial reporting task

```

#include "task_display.h"
#include "shared_state.h"
#include "lab4_config.h"
#include "LcdDisplay.h"
#include <stdio.h>
#include <stdlib.h>

static LcdDisplay lcd(LCD_I2C_ADDR, LCD_COLS, LCD_ROWS);

void vTaskDisplay(void *pvParameters) {
    lcd.init();
    TickType_t xLastWake = xTaskGetTickCount();
    uint8_t reportCounter = 0;
    for (;;) {
        sharedStateLock();
        ActuatorState *s = sharedStateGet();
        bool relayOn = s->relayActualOn;
        float pwmRamp = s->pwmRamped;
        bool alert = s->overloadAlert;
        sharedStateUnlock();
        // LCD update
        char line1[17], line2[17];
        snprintf(line1, 17, "Relay:%-3s PWM%3d%",
            relayOn ? "ON" : "OFF", (int)(pwmRamp + 0.5f));
        snprintf(line2, 17, "Out:%3d%% %s",
            (int)(pwmRamp + 0.5f), alert ? "!ALERT" : " OK ");
    }
}

```

```

    lcd.showTwoLines(line1, line2);
    // Serial report every 2s
    if (++reportCounter >= 4) {
        reportCounter = 0;
        printf("\r\n--- Status Report ---\r\n");
        printf("Relay: %s | PWM: %d%%\r\n",
            relayOn ? "ON" : "OFF", (int)(pwmRamp+0.5f));
        printf("Alert: %s\r\n",
            alert ? "OVERLOAD" : "Normal");
    }
    vTaskDelayUntil(&xLastWake,
        pdMS_TO_TICKS(TASK_DISPLAY_PERIOD_MS));
}
}

```

7.2. Service Layer

Actuator Conditioner

Listing 7.9 – ActuatorConditioner.h — Command conditioning interface

```

#ifndef ACTUATOR_CONDITIONER_H
#define ACTUATOR_CONDITIONER_H
#include "SignalConditioner.h"
class ActuatorConditioner {
public:
    ActuatorConditioner(uint8_t medianWindow, float ewmaAlpha,
        float minClamp, float maxClamp, float maxRampStep);
    float process(float rawCommand);
    float getRampedOutput() const;
    float getConditionedTarget() const;
    const SignalConditioner& getSignalConditioner() const;
    void reset();
private:
    SignalConditioner _conditioner;
    float _maxRampStep, _rampedOutput, _conditionedTarget;
    bool _initialized;
};
#endif

```

Listing 7.10 – ActuatorConditioner.cpp — Conditioning with ramping

```

#include "ActuatorConditioner.h"
#include <math.h>

float ActuatorConditioner::process(float rawCommand) {
    _conditionedTarget = _conditioner.process(rawCommand);
    if (!_initialized) {

```

```

    _rampedOutput = _conditionedTarget;
    _initialized = true;
} else {
    float diff = _conditionedTarget - _rampedOutput;
    if (diff > _maxRampStep) _rampedOutput += _maxRampStep;
    else if (diff < -_maxRampStep) _rampedOutput -= _maxRampStep;
    else _rampedOutput = _conditionedTarget;
}
return _rampedOutput;
}

```

7.3. MCAL Layer

Relay Driver

Listing 7.11 – Relay.h — Relay driver interface

```

#ifndef RELAY_H
#define RELAY_H
#include <Arduino.h>
class Relay {
public:
    Relay(uint8_t pin, bool activeHigh = true);
    void init();
    void turnOn();
    void turnOff();
    void toggle();
    void setState(bool on);
    bool isOn() const;
private:
    uint8_t _pin; bool _activeHigh, _state;
};
#endif

```

Listing 7.12 – Relay.cpp — Relay driver implementation

```

#include "Relay.h"
Relay::Relay(uint8_t pin, bool activeHigh)
    : _pin(pin), _activeHigh(activeHigh), _state(false) {}
void Relay::init() { pinMode(_pin, OUTPUT); turnOff(); }
void Relay::turnOn() {
    _state = true;
    digitalWrite(_pin, _activeHigh ? HIGH : LOW);
}
void Relay::turnOff() {
    _state = false;
    digitalWrite(_pin, _activeHigh ? LOW : HIGH);
}

```

```

void Relay::toggle() { _state ? turnOff() : turnOn(); }
void Relay::setState(bool on) { on ? turnOn() : turnOff(); }
bool Relay::isOn() const { return _state; }

```

PWM Actuator Driver

Listing 7.13 – PwmActuator.h — PWM actuator interface

```

#ifndef PWM_ACTUATOR_H
#define PWM_ACTUATOR_H
#include <Arduino.h>
class PwmActuator {
public:
    PwmActuator(uint8_t pin);
    void init();
    void setDuty(float duty);
    float getDuty() const;
    uint8_t getRawPwm() const;
private:
    uint8_t _pin; float _duty;
};
#endif

```

Listing 7.14 – PwmActuator.cpp — PWM actuator implementation

```

#include "PwmActuator.h"
PwmActuator::PwmActuator(uint8_t pin) : _pin(pin), _duty(0.0f) {}
void PwmActuator::init() { pinMode(_pin, OUTPUT); analogWrite(_pin, 0); }
void PwmActuator::setDuty(float duty) {
    if (duty < 0.0f) duty = 0.0f;
    if (duty > 100.0f) duty = 100.0f;
    _duty = duty;
    analogWrite(_pin, (uint8_t)(_duty * 255.0f / 100.0f));
}
float PwmActuator::getDuty() const { return _duty; }
uint8_t PwmActuator::getRawPwm() const {
    return (uint8_t)(_duty * 255.0f / 100.0f);
}

```